

```

1 clear; clc; close all;
2 % AM THANH 1: AM THANH GOC
3 ten_file = 'giong_noi_cua_toi.wav';
4 try
5     [tin_hieu_goc, Fs] = audioread(ten_file);
6     if size(tin_hieu_goc, 2) > 1
7         tin_hieu_goc = mean(tin_hieu_goc, 2); % Chuyen ve Mono neu la file Stereo
8     end
9     fprintf('Da nap thanh cong file goc: %s (Fs = %d Hz)\n', ten_file, Fs);
10 catch ME
11     fprintf('Loi he thong: %s\n', ME.message);
12     error('Hay chac chan ban da dat file "%s" vao dung folder dang mo cua MATLAB.',
13         ten_file);
14 end
15 L = length(tin_hieu_goc);
16 t = (0:L-1)/Fs;
17 am_thanh_1_goc = tin_hieu_goc;
18 % AM THANH 2: MO PHONG TAI NGUOI GIA NGHE (HOI CHUNG PRESBYCUSIS)
19 N_order = 60;
20 f_cat_lowpass = 1200 / (Fs/2);
21 b_tai = fir1(N_order, f_cat_lowpass, 'low');
22 am_thanh_tan_so_thap = filter(b_tai, 1, am_thanh_1_goc);
23 am_thanh_mo_phong = 0.7 * am_thanh_1_goc + 0.7 * am_thanh_tan_so_thap;
24
25 am_thanh_2_nguoi_gia = am_thanh_mo_phong * 0.3;
26
27 % AM THANH 3: XU LY TRO THINH THONG MINH DE NGUOI GIA NGHE DUOC RO RANG
28 f_cat = [1000, 5000] / (Fs/2);
29
30 b_bandpass = fir1(N_order, f_cat, 'bandpass');
31 1000Hz - 1400Hz
32 am_thanh_dai_boost = filter(b_bandpass, 1, am_thanh_1_goc);
33
34 am_thanh_3_qua_tro_thinh = (am_thanh_1_goc +
35 if max(abs(am_thanh_3_qua_tro_thinh)) > 1
36     am_thanh_3_qua_tro_thinh = am_thanh_3_qua_tro_thinh / max(abs(
37     am_thanh_3_qua_tro_thinh));
38 end
39 % DO THI TRUC QUAN CHUNG MINH 3 GIAI DOAN (MIEN THOI GIAN)
40 figure('Name', 'Do An Mo Phong Lao Thinh Presbycusis', 'Position', [100, 100, 1000,
41     700]);
42 subplot(3,1,1); plot(t, am_thanh_1_goc, 'b');
43 title('1. Am thanh GOC');
44 ylabel('Bien do'); grid on;
45
46 subplot(3,1,2); plot(t, am_thanh_2_nguoi_gia, 'r');
47 title('2. Am thanh NGUOI GIA NGHE ');
48 ylabel('Bien do'); grid on;
49
50 subplot(3,1,3); plot(t, am_thanh_3_qua_tro_thinh, 'g');
51 title('3. Am thanh XU LY TRO THINH ');
52 ylabel('Bien do'); xlabel('Thoi gian (s)'); grid on;
53
54 N_fft = 4096; % So diem FFT
55 F_axis = (0:N_fft/2-1) * (Fs / N_fft); % Truc tan so (Hz)
56
57 % Tinh pho bien do (chuyen sang thang do dB)
58 Spect_1 = 20*log10(abs(fft(am_thanh_1_goc, N_fft)));
59 Spect_2 = 20*log10(abs(fft(am_thanh_2_nguoi_gia, N_fft)));
60 Spect_3 = 20*log10(abs(fft(am_thanh_3_qua_tro_thinh, N_fft)));
61
62 figure('Name', 'Phan Tich Mien Tan So', 'Position', [150, 150, 800, 500]);
63 plot(F_axis, Spect_1(1:N_fft/2), 'b', 'LineWidth', 1.2); hold on;
64 plot(F_axis, Spect_2(1:N_fft/2), 'r', 'LineWidth', 1);
65 plot(F_axis, Spect_3(1:N_fft/2), 'g', 'LineWidth', 1);
66 title('Pho bien do cac tin hieu (Minh chung muc suy hao va bu dap)');
67 xlabel('Tan so (Hz)');
68 ylabel('Bien do (dB)');
69 legend('Am thanh goc', 'Nguoi gia nghe (Suy hao)', 'Sau tro thinh (Da bu dap)');
70 xlim([0 4000]); % Gioi han o 4000Hz de nhin ro vung cat 1000Hz-1400Hz

```

```

70 grid on;
71
72 % PHAT AM THANH SO SANH THEO THU TU (LAN LUOT 1 -> 2 -> 3)
73 thoi_gian_phat = L/Fs;
74 fprintf('\n--- 1. DANG PHAT: AM THANH GOC ---\n');
75 sound(am_thanh_1_goc, Fs);
76 pause(thoi_gian_phat + 1);
77
78 fprintf('--- 2. DANG PHAT: TAI NGUOI GIA NGHE ---\n');
79 sound(am_thanh_2_nguoi_gia, Fs);
80 pause(thoi_gian_phat + 1);
81
82 fprintf('--- 3. DANG PHAT: AM THANH QUA XU LY TRO THINH ---\n');
83 sound(am_thanh_3_qua_tro_thinh, Fs);
84
85
86 save('du_lieu_du_an.mat', 'am_thanh_1_goc', 'am_thanh_3_qua_tro_thinh', 'Fs');

```

Phân tích cấu trúc thuật toán xử lý âm thanh

Phần 1: Xử lý và Chỉnh sửa âm thanh (Cốt lõi thuật toán)

- **Giai đoạn 1 (Âm thanh gốc):** Đọc file .wav của bạn vào hệ thống.
- **Giai đoạn 2 (Phá đi):** Chỉnh âm thanh thành kiểu "người già nghe". Code dùng bộ lọc thông thấp (Low-pass) để chặn các âm bổng (tần số cao), chỉ cho âm trầm đi qua, đồng thời bóp nhỏ âm lượng tổng thể xuống còn 30%.
- **Giai đoạn 3 (Bù đắp lại):** Đóng vai máy trợ thính. Code dùng bộ lọc thông dải (Band-pass) để "tóm" đúng cái khoảng âm bổng vừa bị mất (từ 1000Hz đến 5000Hz), khuếch đại riêng khúc đó lên 19 lần, rồi trộn lại vào âm thanh gốc để người già nghe rõ hơn.

Phần 2: Đồ thị 1 - Phân tích Miền Thời Gian (Time Domain)

Phần này vẽ ra cửa sổ Figure đầu tiên (gồm 3 đồ thị nằm dọc).

- **Nội dung:** Trục ngang là Thời gian (s), trục dọc là Biên độ (độ to nhỏ của tín hiệu).
- **Đồ thị này nói lên điều gì?** Nó cho bạn nhìn thấy hình dáng của "sóng âm" chớp tắt theo thời gian thực. Nhìn vào đây, bạn sẽ dễ dàng thấy: Đồ thị 1 (gốc) dao động mạnh. Đồ thị 2 (người già) sóng xẹp lép (do bị giảm còn 30% âm lượng). Đồ thị 3 (trợ thính) sóng phồng to trở lại.
- **Nhược điểm:** Đồ thị này chỉ cho biết âm thanh to hay nhỏ, chứ KHÔNG cho biết tần số nào (âm trầm hay bổng) đang bị mất. Đó là lý do ta cần phần 3.

Phần 3: Đồ thị 2 - Phân tích Miền Tần Số (Frequency Domain)

Đây là cửa sổ Figure thứ hai (vẽ chồng 3 đường Xanh, Đỏ, Xanh lá lên nhau). Đây là đồ thị quan trọng nhất của bài toán.

- **Nội dung:** Trục ngang là Tần số (Hz) (đại diện cho âm trầm ở bên trái, âm bổng ở bên phải). Trục dọc là Biên độ (dB) (đại diện cho năng lượng/độ to của từng tần số đó).
- **Cách tạo ra nó:** Code phải dùng đến lệnh `fft` (Fast Fourier Transform). Hãy tưởng tượng âm thanh gốc là một "cột sinh tố" thập cẩm. Lệnh FFT là cái máy chất lọc, phân tích xem trong cốc đó có bao nhiêu % dâu (âm bổng), bao nhiêu % chuối (âm trầm) để rải đều lên trục ngang.
- **Đồ thị này nói lên điều gì?** Nó là bằng chứng danh thép nhất cho thấy thuật toán của bạn hoạt động đúng:
 - **Đường Xanh dương (Gốc):** Dàn đều từ 0 đến 4000Hz.
 - **Đường Đỏ (Người già):** Từ 0 đến 1000Hz nó bám khá sát đường gốc (vì âm trầm vẫn nghe được). Nhưng sau mốc 1000Hz, đường đỏ cắm đầu đi xuống rất sâu → Chứng minh tần số cao đã bị suy hao (diết).

- **Đường Xanh lá (Trợ thính):** Tại vùng 1000Hz - 4000Hz, đường xanh lá được đẩy vổng lên cao, vượt qua cả đường đỏ và tiem cận lại đường xanh dương → Chứng minh máy trợ thính đã khuếch đại thành công đúng cái dải tần bị điếc.

Phần 4: Phát âm thanh ra loa

Phần cuối cùng dùng lệnh `sound` để phát lần lượt 3 âm thanh đã xử lý ở Phần 1. Nhờ có các hàm `pause`, bạn sẽ được nghe theo thứ tự: Nghe gốc → Nghe kiểu bị điếc → Nghe sau khi đeo trợ thính để tai bạn tự kiểm chứng lại những gì mắt bạn vừa nhìn thấy ở 2 phần đồ thị.

Bạn có muốn mình giải thích sâu hơn một chút về phép toán `fft` (cách nó tách sóng âm thành các tần số) ở đồ thị thứ 2 không, hay bạn muốn tìm hiểu cách tinh chỉnh lại các con số bù đắp âm thanh?

Các hàm chủ đạo trong code

1. Hàm chủ đạo cho xử lý âm thanh (Phần 1)

Đây là "bộ não" của hệ thống, nơi bạn thay đổi đặc tính âm thanh:

- **audioread:** Hàm "cổng vào", dùng để đọc file âm thanh từ ổ cứng và chuyển thành mảng số để máy tính hiểu được.
- **fir1:** Hàm "thiết kế bộ lọc". Đây là hàm quan trọng nhất để tạo ra các bộ lọc (Low-pass, Band-pass). Bạn đưa cho nó bậc bộ lọc (`N_order`) và tần số cắt, nó sẽ trả về các hệ số lọc.
- **filter:** Hàm "thi công". Sau khi đã có bộ lọc từ `fir1`, bạn dùng `filter` để áp dụng nó vào tín hiệu gốc. Nó thực hiện phép toán tích chập để loại bỏ hoặc giữ lại các dải tần mong muốn.

2. Hàm chủ đạo cho phân tích miền thời gian (Phần 2)

Phần này đơn giản là dùng để trực quan hóa dữ liệu:

- **plot:** Đây là hàm vẽ đồ thị cơ bản nhất. Trong phần này, nó vẽ mối quan hệ giữa thời gian (trục x) và biên độ (trục y).
- **subplot:** Hàm dùng để chia màn hình đồ thị thành nhiều ô (ở đây là 3 ô xếp dọc), giúp so sánh các giai đoạn (Gốc - Già - Trợ thính) trên cùng một kích thước trục để người xem dễ đối chiếu.

3. Hàm chủ đạo cho phân tích miền tần số (Phần 3)

Đây là phần đòi hỏi kiến thức toán học cao cấp hơn một chút:

- **fft (Fast Fourier Transform):** Đây là hàm chủ đạo quan trọng nhất của toàn bộ bài toán phân tích tần số. Nó là một thuật toán biến đổi tín hiệu từ miền thời gian (nơi bạn chỉ thấy biên độ thay đổi) sang miền tần số (nơi bạn thấy các thành phần tần số cụ thể như Bass, Mid, Treble).
- **abs & log10:** Sau khi `fft` cho ra các số phức, bạn dùng `abs` để lấy độ lớn biên độ và `log10` để chuyển đổi đơn vị sang Decibel (dB). Tai người nghe âm thanh theo logarit, nên thang đo dB phản ánh đúng thực tế cảm nhận của tai người hơn.

4. Hàm chủ đạo cho phát âm thanh (Phần 4)

- **sound:** Hàm này đơn giản nhưng cực kỳ hiệu quả. Nó gửi mảng dữ liệu đã xử lý vào thiết bị xuất âm thanh (loa/tai nghe) với tốc độ `Fs` (tần số lấy mẫu) đã định sẵn.
- **pause:** Dùng để kiểm soát dòng thời gian, giúp bạn nghe được các file âm thanh lần lượt mà không bị chồng lấp, tạo khoảng nghỉ cần thiết để não bộ kịp phân biệt sự khác nhau giữa các âm thanh.

Tóm tắt nhanh cho bạn:

- Xử lý: `fir1 + filter`
- Phân tích tần số: `fft`
- Trực quan hóa: `plot + subplot`